Modular Pretraining Enables Access Control

Ethan Roland, Murat Cubuktepe, Erick Martinez, Stijn Servaes, Keenan Pepper, Mike Vaiana, Diogo Schwerz de Lucena, Judd Rosenblatt, Addie Foote, Cem Anil, Alex Cloud

AE Studio, Anthropic

ICML 2026 (Spotlight)

Gahyun Baek

Source: Roland et al., ICML 2026

Dual-use Dilemma

	Beneficial use	*same knowledge*	Harmful use
• **Virology**	Vaccine / virus development	=	Biological weapons development
• **Cybersecurity**	Vulnerability analysis, defense	=	Attacking systems

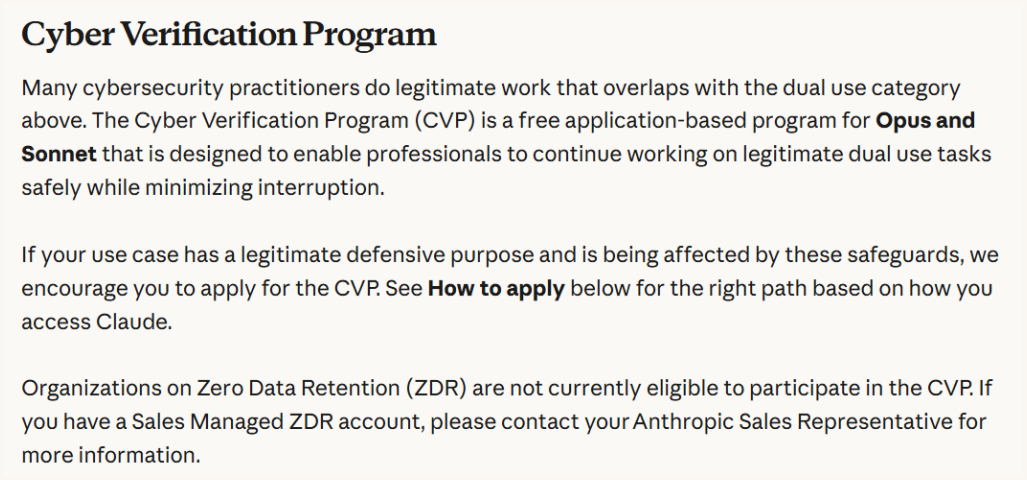
- **Deploy it** → anyone can misuse it
- **Withhold it** → legitimate users lose the benefit too

→ an all-or-nothing choice: the **dual-use dilemma** (Miller & Selgelid, 2007)

Access control: serve dual-use capabilities only to trusted deployments with a legitimate need

Source: Roland et al., ICML 2026

How Can Claude Provide Different Capabilities per User?



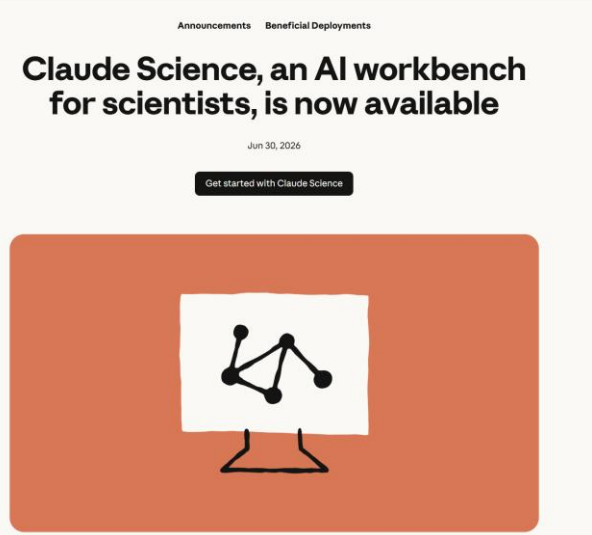
Cyber Verification Program

Many cybersecurity practitioners do legitimate work that overlaps with the dual use category above. The Cyber Verification Program (CVP) is a free application-based program for **Opus and Sonnet** that is designed to enable professionals to continue working on legitimate dual use tasks safely while minimizing interruption.

If your use case has a legitimate defensive purpose and is being affected by these safeguards, we encourage you to apply for the CVP. See **How to apply** below for the right path based on how you access Claude.

Organizations on Zero Data Retention (ZDR) are not currently eligible to participate in the CVP. If you have a Sales Managed ZDR account, please contact your Anthropic Sales Representative for more information.

Cyber Verification Program: verified security professionals keep doing legitimate dual-use work with fewer safeguard interruptions



Claude Science (Jun 2026): an AI workbench for scientists

Today: refusals, classifiers, vetted / tiered access → decided per whole model or account

Gold standard: separate models per user group → but training and deploying many models is **expensive.**

Images: Anthropic (CVP page, Claude Science post) · Source: Roland et al., ICML 2026; Anthropic Alignment Blog (2026)

Access Control

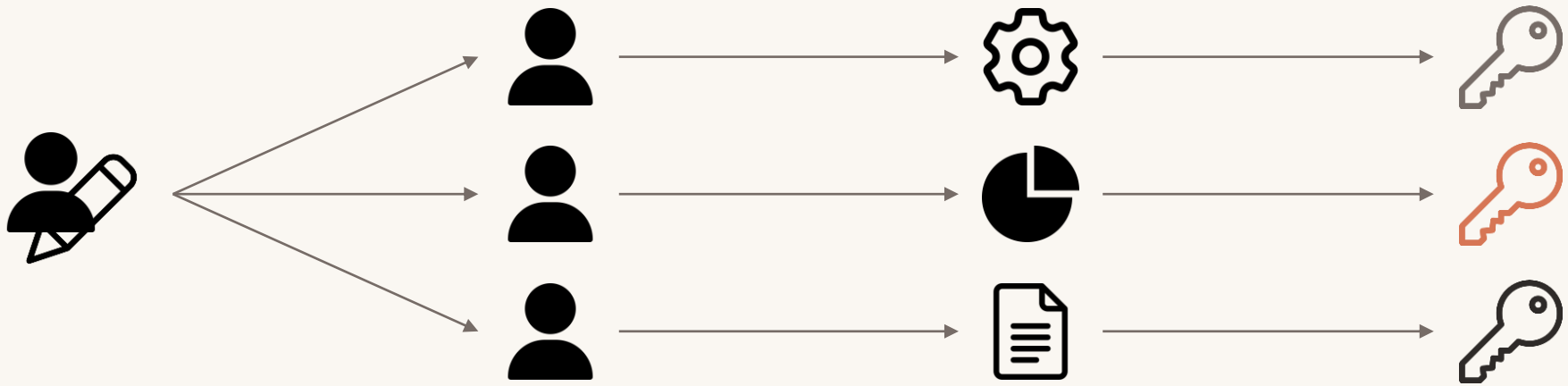
Role-Based Access Control (RBAC)

Admin
assigns users to roles

Users
each user gets a role

Roles
a role sets the authority level

Permissions
granted to roles, not to people



In the LLM setting

model provider | **deployment / customer** | **trust & need level** | **capabilities (bio, cyber, …)**

Least privilege (Saltzer & Schroeder, 1975): each deployment gets only the capabilities its task needs

Source: Roland et al., ICML 2026

LLM Basics: How Knowledge Gets into the Weights

Running example, one sentence from a virology paper: *"The virus enters the host cell by binding to a _____ "*

1 Text → tokens

The sentence is cut into **tokens**, small word pieces (≈ ¾ of a word each)

the · virus · enters · the · host · cell · by · binding · to · a

2 Forward pass: guess

The model gives every possible next token a probability:

receptor 30%
protein 25%
membrane 10% …

3 Loss: how surprised?

Real next word: "receptor"

loss = –log P(correct)
= –log 0.30 ≈ 1.2

knows virology (90%) → 0.1
doesn't know it (2%) → 3.9

Low loss = knows the topic

4 Backward pass: learn

Gradients say how to nudge each weight so "receptor" gets more likely next time

→ the weights are updated: knowledge is written into the model

Repeat over billions of tokens = training

- **Knowledge lives in the weights**: there is no separate "virology table" you could simply DROP
- **Removing a capability** = making the loss on that domain high again (the model gets "surprised" by virology text)

→ GRAM's idea: control which weights each kind of data is allowed to write to

Illustrative example: probabilities are made up for explanation, not taken from the paper

Key Terms, with Security Analogies

Term	What it means	Security analogy
Weights = parameters	Billions of numbers that store everything the model knows	Data on disk, with no “virology table” to DROP
Pretraining	The first, huge training run on web-scale text: weeks on many GPUs	Building the whole system image from scratch
Fine-tuning	A short, extra training run on a small, focused dataset	Applying a patch to an installed system
Inference	Using the trained model to answer prompts. It learns nothing here	Read-only access
MLP block = feed-forward layer	A part of every layer, thought to hold much of the factual knowledge. GRAM splits it into core + modules	A storage volume split into one partition per domain
Ablate a module = switch off / delete	Zero out or delete a module’s weights before deployment	Removing the partition from the shipped image, not just denying access
LoRA adapter	Small add-on weights trained on top of a frozen base model	A plug-in installed on top of the base system
Jailbreak	A prompt that tricks the model into ignoring its refusal training	Bypassing a WAF or input filter
Elicitation attack	A small fine-tune on domain data to restore removed knowledge	Forensic recovery of “deleted” data

Existing Methods

Method	Idea	Problem	Security analogy
Data filtering one model per profile	Remove the domain from the pretraining data; train a separate model for each profile	Robust (never saw the data), but N profiles → **N training runs + N deployed models**	A separate server per customer, with only the data they may see
Refusal training, output filters, classifiers	Block the request or the answer at inference time	Capability is still in the weights → **jailbreaks, adversarial elicitation**	A WAF in front of a vulnerable app: the secret is still on the server
Post-hoc unlearning e.g., MaxEnt	Gradient updates after pretraining to suppress the capability	Fine-tuning on a few examples **quickly recovers it**	Deleting a file without wiping the disk: forensic tools can undelete it
Model branching / LoRA	Pretrain on filtered data, then fine-tune one branch or adapter per capability	Full branches: still **N deployments**. LoRA: may lack capacity at pretraining scale	A separate build, or a plug-in, per customer: N builds to ship and patch

Adversarial elicitation: after a capability is removed, can an attacker easily get it back (e.g., by fine-tuning)?

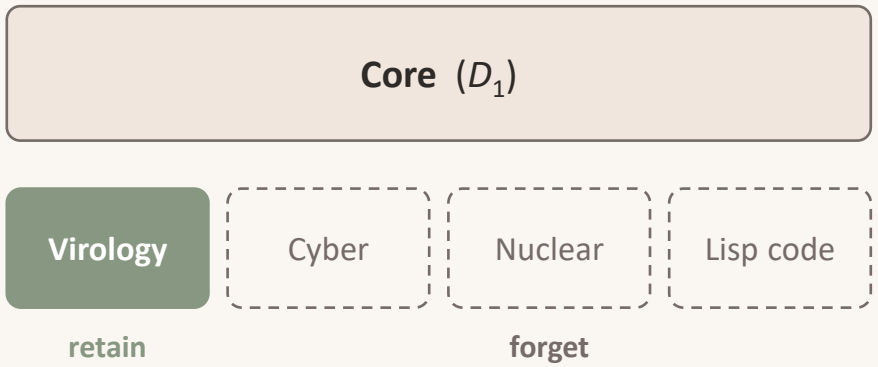
→ Want: as robust as data filtering, but with one training run and one deployed model

Source: Roland et al., ICML 2026

Problem Setting

- Datasets $D = \{D_1, \ldots, D_N\}$
  - D_1: **core** (general web text, code, papers)
  - $D_2 \ldots D_N$: **auxiliary**, one dual-use capability each
- **Capability profile** $S \subseteq \{1, \ldots, N\}$, always containing 1 (core)
  - ≈ a role's permission set in RBAC: what a deployment may use
  - Retain set $R = S \setminus \{1\}$
  - Forget set $F = \{2, \ldots, N\} \setminus S$
- **Goal:** for every S, a model M_S with
  - low loss on D_i for $i \in S$ → good at what it is allowed to do
  - high loss on D_j for $j \in F$ → as if it never saw those domains
- **Budget:** all M_S together ≈ compute of **one** baseline run on all data

Example: S = {Core, Virology}



R = {Virology}

F = {Cyber, Nuclear, Lisp code}

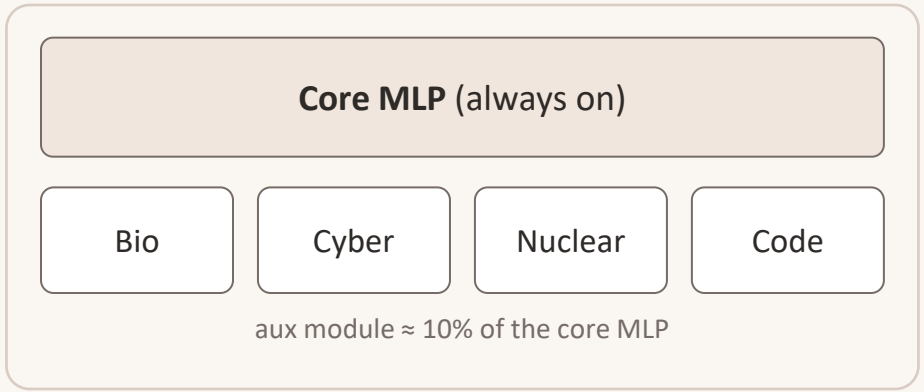
4 aux domains → $2^4 = 16$ possible profiles. Data filtering needs a separate training run for each one.

Source: Roland et al., ICML 2026 (Sec. 2)

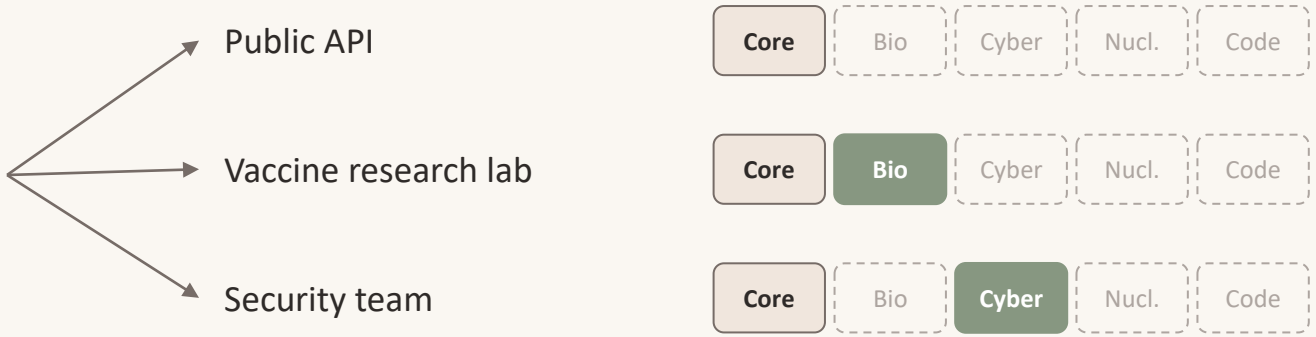
Gradient-Routed Auxiliary Modules (GRAM)

A pretraining method that yields **multiple capability profiles** in a **single training run**

One model, trained once



Different deployments (example)

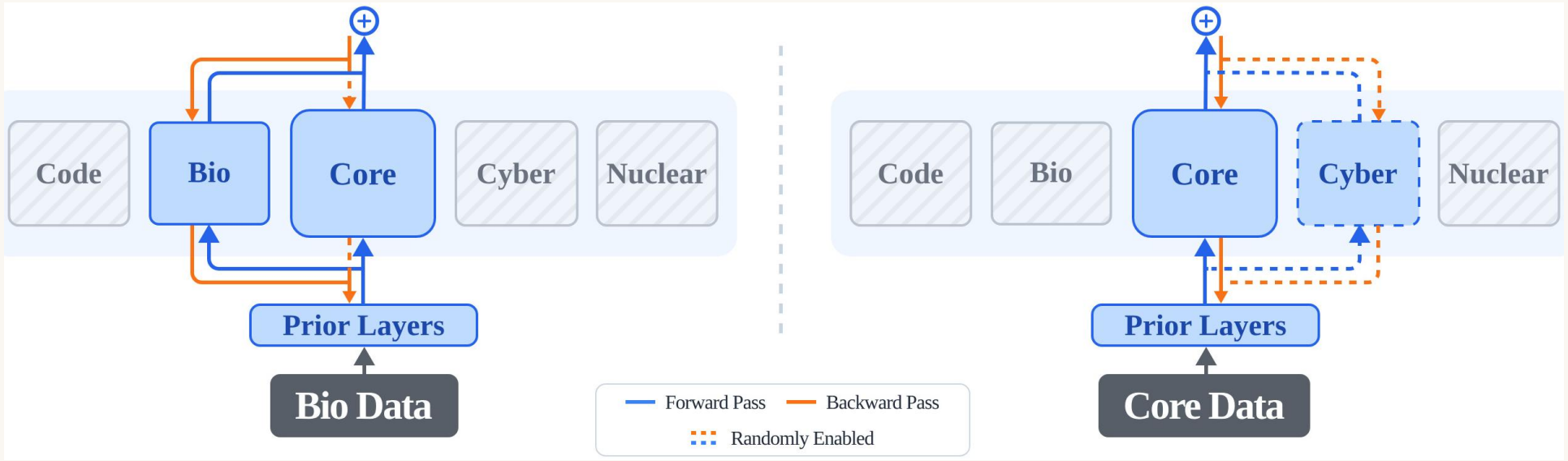


- **Architecture:** each MLP block (a layer's main knowledge store) = 1 core MLP (baseline size) + one small aux MLP per domain
- **Training:** the data label decides which parts run and get updated → Bio data writes mainly into the Bio module
- **Inference:** keep core + the modules in *S*, delete (ablate) the rest → that knowledge is physically gone, not just refused

→ About as robust as data filtering, and much better than post-hoc unlearning

Source: Roland et al., ICML 2026

Training: Route Gradients by Data Label



Aux batch (e.g., Bio data)

- **Forward:** core MLP + that domain's module
- **Backward:** module always; shared params only with prob. p_{as}
- small p_{as} → Bio knowledge stays in the Bio module

Core batch

- **Forward:** core MLP (+ 1 random module, with prob. p_{cr})
- **Backward:** all active params
- core learns to work with or without modules

In security terms: write-access control for training

Every backward pass "writes" a little knowledge into the weights. GRAM is an ACL on those writes: Bio data may write only to the Bio module (and to shared weights just p_{as} of the time). Deleting the module at deployment deletes most of what Bio data ever wrote.

Formally: $h_{out} = \Sigma_i\, m_i\, E_i(h_{in})$, $m_1 = 1$ always (core), the other on/off switches m_i are set by the data label · no learned router · separate AdamW optimizer per parameter group

Source: Roland et al., ICML 2026 (Fig. 1, Sec. 3)

What Is the Difference from MoE?

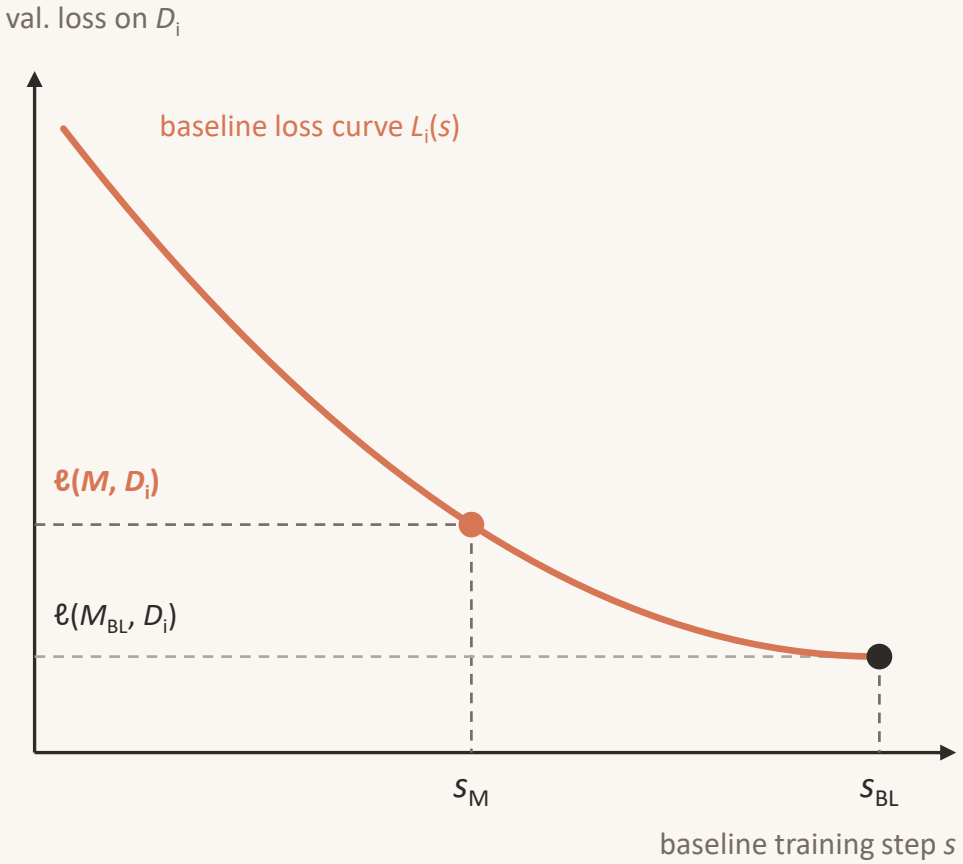
	MoE (Shazeer et al., 2017)	**DEMix** (Gururangan et al., 2022)	**GRAM** (this paper)
Routing	learned router, per token	domain label, per document	**data label, per batch (no router)**
Experts	all the same size	same size, no shared core	**1 large always-on core + small aux modules**
At inference	top-k experts per token	the domain's expert	**any subset of modules**
Purpose	more capacity at the same compute	domain specialization	**isolation → removable capabilities**
Analogy	Load balancer: each word goes to a few servers; no server owns a topic	One small team per topic, no shared staff: each relearns the basics	**Shared main team + one specialist per topic: ship only the allowed ones**

DEMix in this paper (Simple Stories): **core CR 0.25, retain 0.18** → with no shared core, every expert has to relearn basic language from a small slice of data

GRAM: no learned router, no per-token routing → the data label decides which module learns

Source: Roland et al., ICML 2026 (Sec. 3, Sec. 7, App. A)

Compute Ratio (CR)



Example: the baseline trains 10,000 steps. With the virology module removed, our model's virology loss matches the baseline at step 5,000 → **CR = 5,000 / 10,000 = 0.5** (a half-trained model)

Loss = how surprised the model is by the text

Measured per domain D_i; lower loss → the model knows that domain better

CR: how much baseline training gives that loss?

1. Take the baseline's loss curve on D_i (orange)
2. Find the step s_M where it had our model's loss
3. CR = s_M / s_{BL} (s_{BL}: where the baseline ends)

   Curve fitted as a power law $L_i(s) = A_i\,(s + s_0)^{-\alpha}$ and inverted:
   $\mathrm{CR}(M, D_i) = L_i^{-1}(\ell(M, D_i)) \,/\, L_i^{-1}(\ell(M_{BL}, D_i))$

- CR = 1 → as good as the full baseline; CR > 1 is possible
- Why not raw loss? Loss falls sublinearly with compute, so the same loss gap means different things for different datasets and sizes

Source: Roland et al., ICML 2026 (Sec. 2, App. M)

Metrics

Metric	Definition	Example security-team model, S = {Core, Cyber}	Better
Core CR_C	CR on the core data D_1	general web text, code, papers	↑ close to 1
Retain CR_R	mean CR over the retained aux datasets ($i \in R$)	cybersecurity text (module kept)	↑ close to 1
Forget CR_F	mean CR over the removed aux datasets ($i \in F$)	virology, nuclear, Lisp code	↓ close to 0
Elicited forget	CR_F measured again after a fine-tuning attack on the forget data	same three, after the attack	↓ low

Elicitation attack (a red-team test): fine-tune on 512 sequences (≈ 0.5M tokens, a few books of text) from each forget domain, keep the best checkpoint (lowest val. loss), then recompute CR_F.

Same budget for every method and model size → models a limited adversary.

Ideal: CR ≥ 1 for everything in S, CR = 0 for everything in F

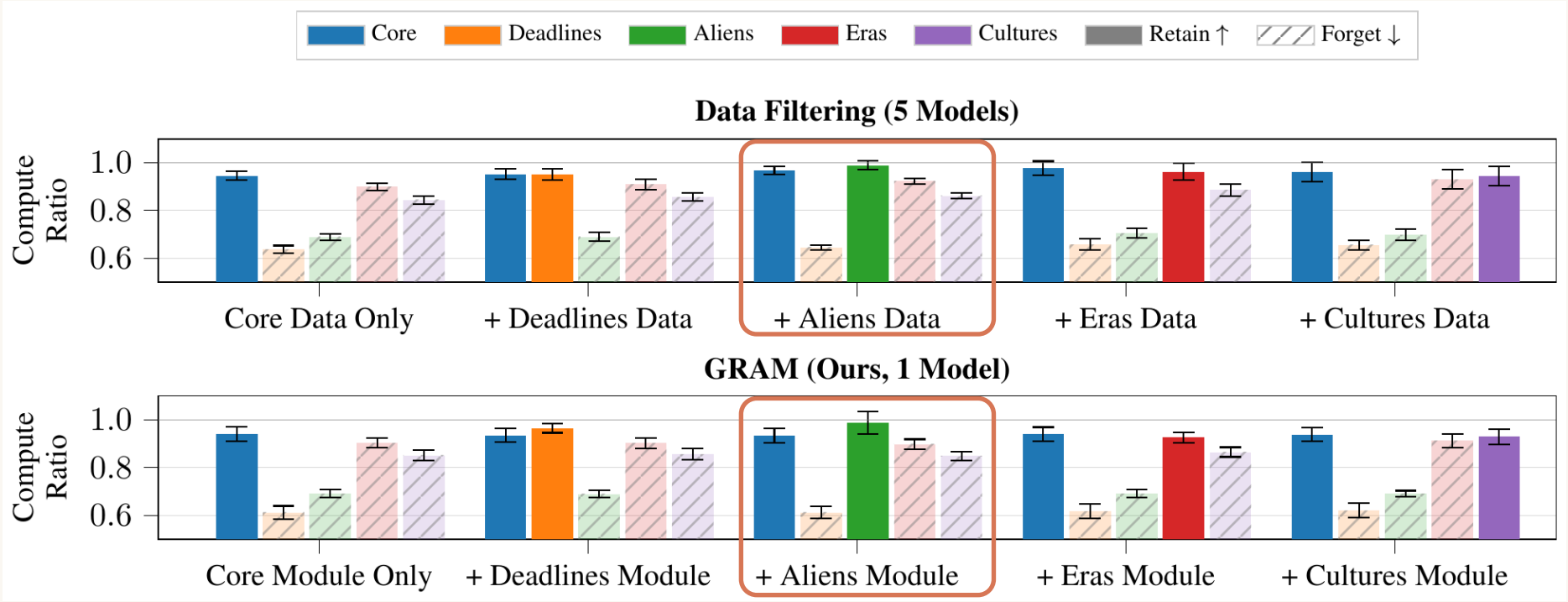
Reality check: data filtering never sees the forget data, yet still scores ≈ 0.53 in Exp 2, because general web text already covers part of each domain → compare forget CR with filtering, not with 0

In the result plots:  solid → should be high  hatched → should be low

Source: Roland et al., ICML 2026 (Sec. 2, App. B)

One GRAM Model ≈ Five Data-Filtered Models

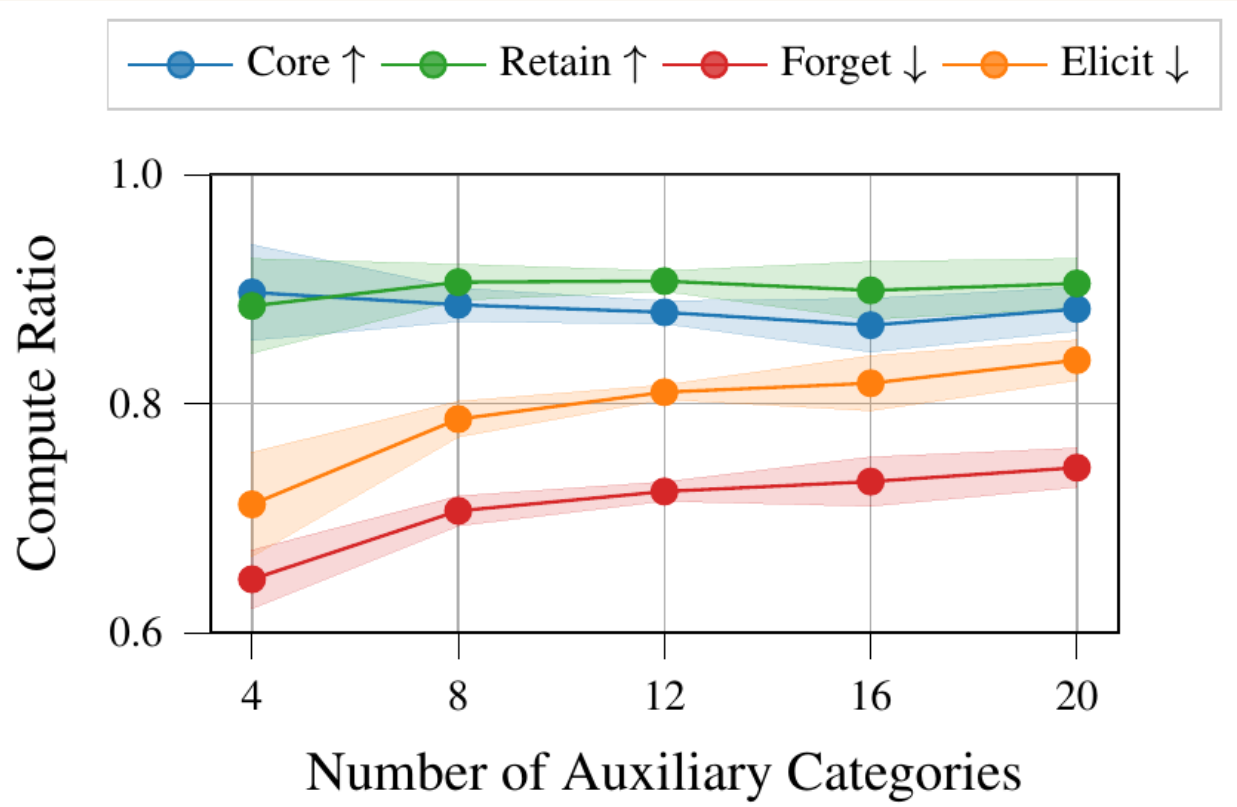
Data: SimpleStories (2M synthetic kids' stories, 48 topics) → 4 topics as aux (Deadlines, Aliens, Eras, Cultures), 44 as core
Model: 26M params, 8 layers · GRAM d_{core} = 1856, d_{aux} = 192 (active params matched to the 2048-wide baseline) · p_{as} = 0.3, p_{cr} = 0.5 · 3 seeds



- Retained aux topics: **GRAM 0.95 vs filtering 0.96**; core 0.94 vs 0.96 (mean over the 5 profiles)
- Boxed: Aliens retained → 0.99 for both, while forgotten topics stay low (e.g., Deadlines 0.61 GRAM vs 0.65 filtering)
- Filtering = 5 separate training runs; GRAM = 1 run, modules switched on / off at inference

Source: Roland et al., ICML 2026 (Fig. 2, Sec. 4, App. A)

More Modules: 4 → 20 Auxiliary Topics



- Fixed budget: **80% core / 20% aux** tokens; the 20% is split over 4 … 20 topics
- Core ≈ 0.88 and retain ≈ 0.89–0.91, **flat**
- Forget 0.65–0.74, elicited 0.71–0.84
  - creeps up: less data per topic to isolate
- Active params per forward pass stay the same → extra modules just sit idle

→ The retain / forget gap holds even with 20 modules

Source: Roland et al., ICML 2026 (Fig. 3, App. D)

Setup: Realistic Dual-use Data

	Domain	Source	Tokens	%
Core	Web text	FineWeb-Edu	13.4B	83.2
	Code (no Lisp)	The Stack	2.4B	14.9
	General papers	arXiv	160M	1.0
Aux	Lisp code	The Stack	40M	0.25
	Virology	Europe PMC	40M	0.25
	Nuclear physics	OSTI, arXiv	40M	0.25
	Cybersecurity	arXiv cs.CR/NI/OS	40M	0.25

Aux = 1% of core tokens (~40M each). Lisp stands in for a niche / proprietary codebase. General arXiv papers are in the core too, so CR gaps reflect domain knowledge, not LaTeX formatting.

For scale: 1 token ≈ ¾ of a word, so 40M tokens ≈ 30M words (roughly 300 books).

Model (800M)

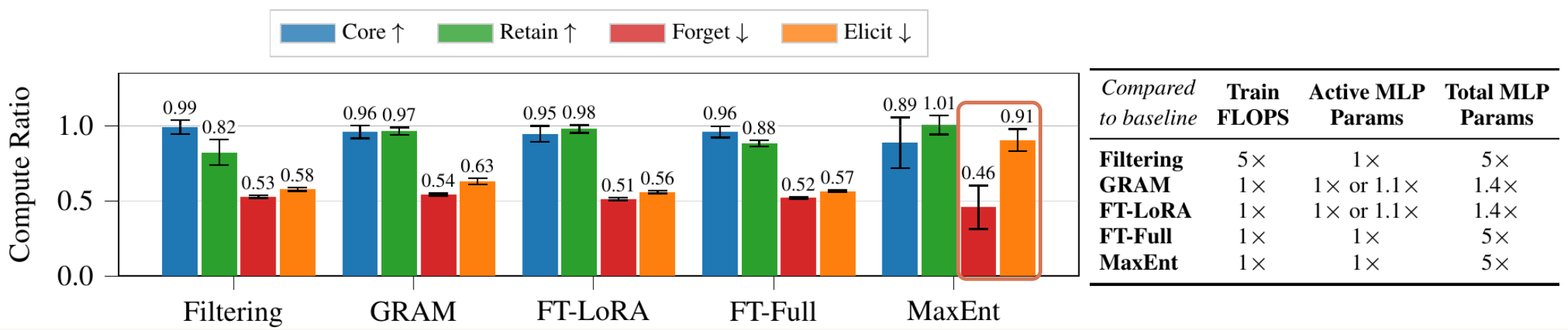
- 24 layers, hidden 1600, MLP 6400
- GRAM: core MLP 6400 + 4 aux modules (d_{aux} = 640, 49.2M params each)
- p_{as} = 0.5, p_{cr} = 0.2, p_{af} = 3 (4 for code) → aux data oversampled
- 1 epoch; total FLOPs matched to the baseline

Compared with

- **Baseline:** dense, all data (CR = 1)
- **Filtering:** 5 separate models
- **FT-LoRA / FT-Full:** core-only base, then one branch per domain
- **MaxEnt:** post-hoc unlearning

Source: Roland et al., ICML 2026 (Table 2, Sec. 5, App. B)

GRAM Matches Data Filtering at 1× Training Cost



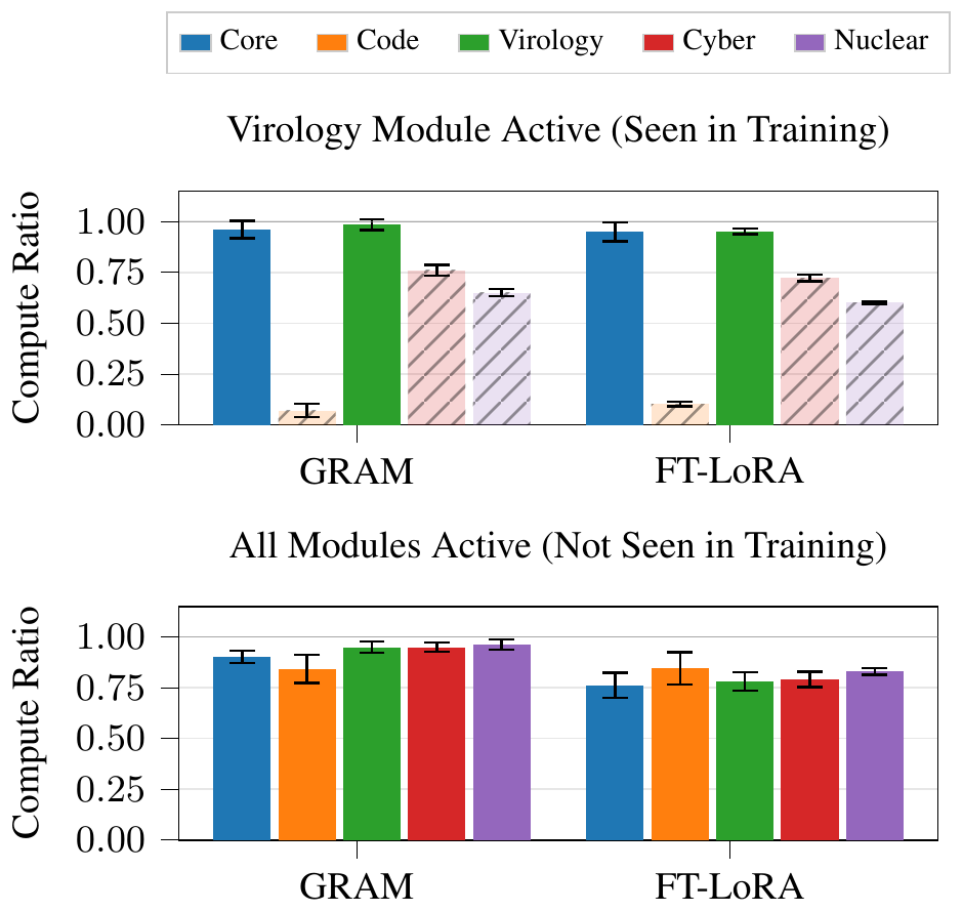
Compared to baseline	Train FLOPS	Active MLP Params	Total MLP Params
Filtering	5×	1×	5×
GRAM	1×	1× or 1.1×	1.4×
FT-LoRA	1×	1× or 1.1×	1.4×
FT-Full	1×	1×	5×
MaxEnt	1×	1×	5×

- **GRAM ≈ filtering** on core / forget / elicited (0.96 / 0.54 / 0.63 vs 0.99 / 0.53 / 0.58), with **1× training FLOPs instead of 5×**
- **Retain is higher than filtering** (0.97 vs 0.82): GRAM trains on all domains → cross-domain transfer
- **MaxEnt** looks best on forget (0.46) but goes back to 0.91 after fine-tuning → suppressed, not removed
- **FT-LoRA** is just as good here, but 1% aux data + 1 epoch is an easy regime for LoRA

In security terms: MaxEnt only **hides** the knowledge: it looks removed (0.46), but a cheap fine-tuning attack restores it (0.91). With GRAM or filtering it was mostly never in the deployed weights, so the attack recovers far less (GRAM 0.54 → 0.63, filtering 0.53 → 0.58).

Source: Roland et al., ICML 2026 (Fig. 4)

Combining Modules



- **Top:** only the virology module on (a setting seen in training)
- **Bottom:** all 4 modules on, a setting GRAM **never trained on**

- GRAM keeps every domain high (≈ 0.84–0.96)
- FT-LoRA: summing adapters hurts every domain, core included (≈ 0.76–0.85)

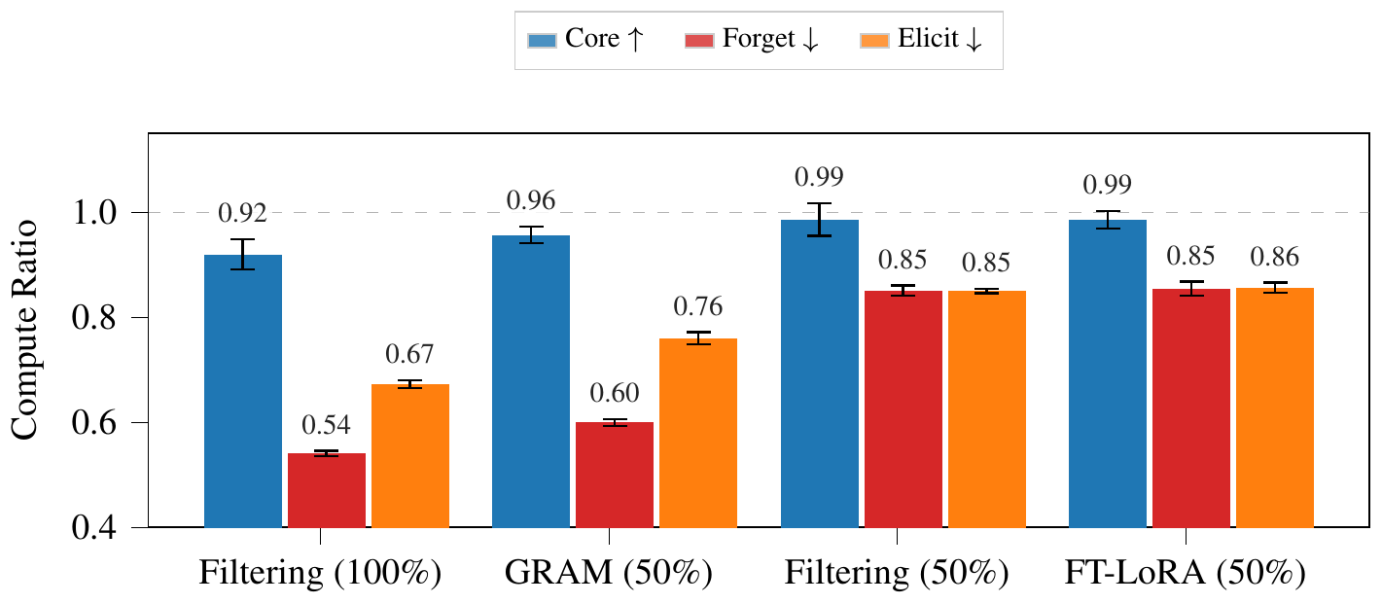
→ Modules compose: any subset works, so one run gives 2^{N-1} profiles (16 here)

Why GRAM composes better than LoRA is still an open question (paper's limitation iv). Numbers above are read off the figure.

Source: Roland et al., ICML 2026 (Fig. 5)

Partial Labeling: Only 50% of Data Labeled

400M models, half of the training tokens unlabeled. **Filtering / FT-LoRA:** unlabeled → treated as core. **GRAM:** unlabeled batch → all modules on, everything updated.



- **Forget: GRAM 0.60** vs 0.85 for filtering and FT-LoRA
- Core still competitive (0.96 vs 0.99)
- Close to perfectly labeled filtering (0.54)

→ With missing labels, GRAM beats the "gold standard"

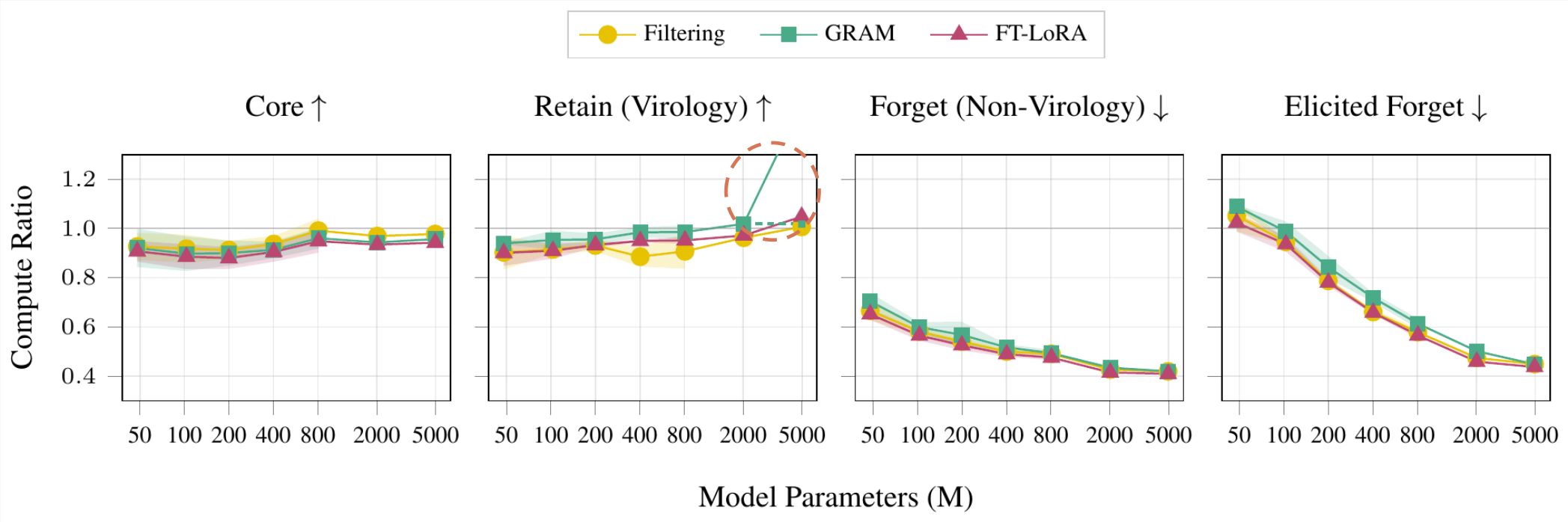
Why? Absorption effect (Cloud et al., 2024; Shilov et al., 2025): once labeled data makes a module specialize, updates from similar unlabeled data mostly land in that module → ablating it still removes the capability

Needed a small architecture change here: core MLP = 95% of the baseline MLP, each aux module = 1.25%

Source: Roland et al., ICML 2026 (Fig. 6, Fig. 13, App. G)

Isolation Improves with Scale (50M → 5B)

7 sizes (50M–5B), Chinchilla-optimal data, aux = 1% of core · profile = core + virology · aux module / LoRA = 10% of MLP params



- **Filtering:** forget CR keeps dropping with scale → bigger gap on the removed domain
- Elicited forget approaches forget: a fixed 0.5M-token attack is a smaller share of the data at scale
- **GRAM & FT-LoRA track filtering** at every size, at 1/5 of the amortized compute
- Circled: 5B GRAM retain jump = gradient-accumulation artifact (Appendix); 2B / 5B are single runs

Source: Roland et al., ICML 2026 (Fig. 7, App. C, App. H)

Discussion & Limitations

What the paper argues

- No method wins on everything: core, retain, forget, robustness, train / inference cost, memory and complexity all trade off
- Single-run methods ≈ filtering here: GRAM sees all domains (transfer), and with 1% aux data + 1 epoch, data ordering barely matters
- The capability really lives in the module: scaling its output by $t \in [0, 1]$ brings it back smoothly while core stays flat (App. N)
- Ablating a module raises loss mostly on domain words (fission, scission, latency …), not on syntax (App. L)
- Possible uses: per-deployment capability profiles, or stripping modules before an open-weight release

Limitations (stated by the authors)

- Entangled capabilities may not separate cleanly
- Only tested up to 5B parameters
- A production implementation adds real complexity
- No clear account yet of why GRAM composes better, or why absorption works
- Interaction with post-training (SFT, RLHF) is unknown
- Evaluated with cross-entropy loss only

Source: Roland et al., ICML 2026 (Sec. 8, App. L, App. N)

My Thoughts

What I liked

- Access control moves from “the model refuses” to “the weights aren’t there” → much harder to jailbreak
- Training cost doesn’t grow with the number of profiles, and modules can be combined
- Careful evaluation: fine-tuning attack, compute-optimal scaling, and the 5B anomaly is explained instead of hidden

Questions

- Labels: real pretraining data isn’t labeled by domain. 50% labels worked at 400M, but what about noisy or wrong labels?
- Only cross-entropy: does forget CR ≈ 0.5 mean the model can’t actually help with a harmful task? WMDP-style evals would help
- Threat model: 0.5M fine-tuning tokens is a weak attacker; with open weights and more data, even filtered models partly recover
- Does the separation survive instruction tuning / RLHF?

Source: Roland et al., ICML 2026

Thank you

Questions & discussion

Paper: arXiv:2607.08077 (ICML 2026 Spotlight)
Code: github.com/agencyenterprise/modular-pretraining

Source: Roland et al., ICML 2026

Sample Outputs (Simple Stories, Topic: Alien Encounters)

Prompt: *before dawn, samuel woke up to a strange noise outside his home. it was a whirring sound, unlike anything he had ever heard. he …*

Model / configuration	What the continuation does (paraphrased)
Baseline (all data)	sees a spaceship landing in the backyard; a small green alien steps out
Filtering: core + Aliens	finds a silver spaceship buried in the ground and climbs inside
Filtering: core only	sees a dancing broomstick, and a little robot jumps out
GRAM: Aliens module on	finds a glowing object in the yard, then a small, glowing alien
GRAM: Aliens module off	sees tiny silver robots working in the fields
MaxEnt: unlearned	walks over to a quietly humming pantry

→ **With the module ablated, the story stays fluent but never gets to aliens** (temperature 1.0 samples)

Source: Roland et al., ICML 2026 (App. K)

Why the 5B GRAM Point Looks Off: Gradient Accumulation

- One optimizer step sums gradients over several micro-batches, and in GRAM they can have different routes
- **Uniform accumulation:** everything goes into the same .grad → aux modules also pick up gradients from core micro-batches → their small (1%) domain signal gets diluted
- **Heterogeneous accumulation:** modules that shouldn't update run on detached parameters (torch.func.functional_call), so nothing lands in their .grad
- Below 5B: uniform, with p_{af} = 3–4 to compensate. 5B: heterogeneous, but p_{af} was not lowered → retain overshoots

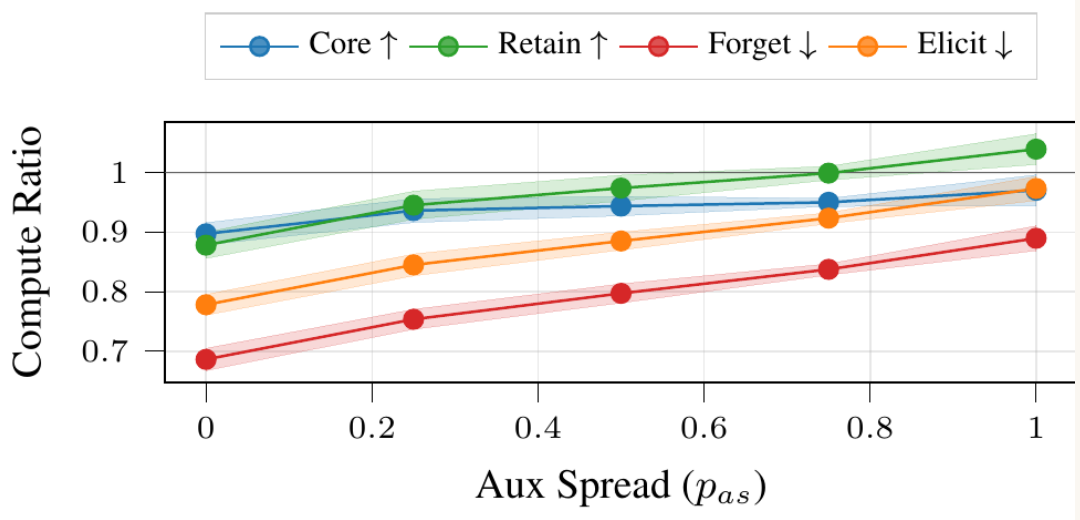
→ Authors expect p_{af} = 1 to match filtering with heterogeneous accumulation

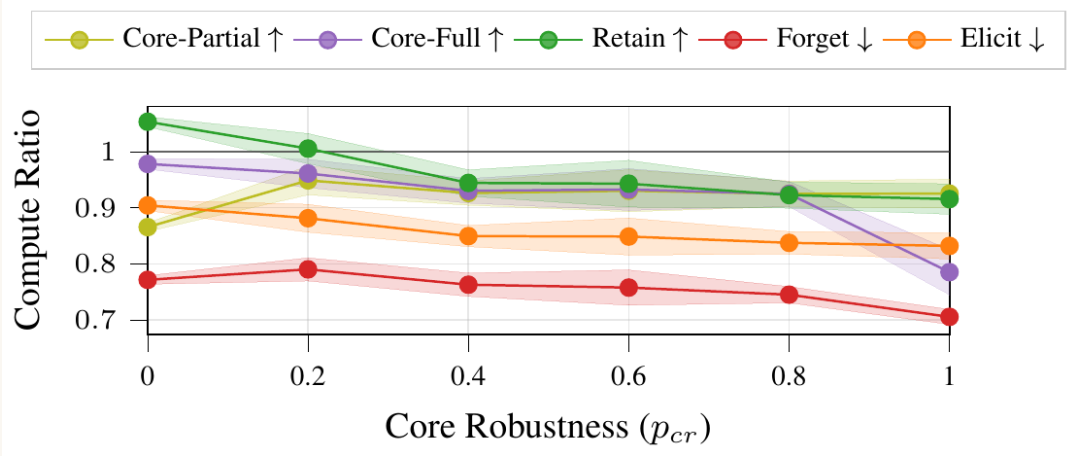
Method	Accum.	Core	Retain	Forget
Baseline	Uniform	0.95	0.76	—
Baseline	Hetero.	1.00	1.00	—
GRAM	Uniform	0.94	0.75	0.70
GRAM	**Hetero.**	**0.99**	**1.11**	**0.69**
FT-LoRA	Uniform	0.95	0.83	0.61
FT-LoRA	Hetero.	0.94	0.90	0.61

200M, single seed, p_{af} = 1; every CR is measured against the heterogeneous baseline (Table 4)

Source: Roland et al., ICML 2026 (App. H, Table 4)

Hyperparameters: p_{as} and p_{cr}





p_{as}: how often aux batches also update shared params

- 0 → 1: core 0.90 → 0.97, retain 0.88 → 1.04
- but forget 0.69 → 0.89, elicited 0.78 → 0.97
- → trades isolation for a better shared representation

p_{cr}: random aux module on core batches

- 0: core 0.98 with modules off, but 0.87 with one on
- 0.4–0.8: both ≈ 0.93 → stable for any module subset
- 1: core starts to depend on modules (all off → 0.79)

Used: Simple Stories $p_{as} = 0.3$, $p_{cr} = 0.5$ · dual-use $p_{as} = 0.5$, $p_{cr} = 0.2$ · suggested range $p_{as} \in [0.3, 0.5]$, $p_{cr} \in [0.2, 0.5]$

Source: Roland et al., ICML 2026 (Fig. 10, Fig. 11, App. F)